



Exercise 1: Mutual Exclusion and Synchronization with Semaphores

Lappeenranta–Lahti University of Technology LUT

Computational Science and Artificial Intelligence

CT30A3350 Operating Systems and System Programming (Lahti)

Date: 15.04.2026

Md Hamidur Rahman Khan, 002475552

Examiner: Ankit Agrawal, PhD

Task 1

The output of x and o were interleaved, and the order changed from one run to another. From figure 1, it can be seen that sometimes xx and oo appeared together, and also in some other cases x and o were mixed. The sequence was not fixed.

This happens because the two processes run at the same time, and there is no semaphore to coordinate their access. Each process prints to the terminal independently, while the CPU scheduler decides which one gets to run at any moment. The sleep statements add random delays and make the output order even less predictable. Therefore, even though the total number of printed characters is correct, the order of x and o is not fixed.

```
hamidur@DESKTOP-7CGRQ8S:~/LUT_exercises/os_sp/ex1/task1$ gcc -o no_sem no_sem.c
hamidur@DESKTOP-7CGRQ8S:~/LUT_exercises/os_sp/ex1/task1$ ./no_sem & ./no_sem o
[1] 7792
xxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxo[1]+ Done                      ./no_sem
hamidur@DESKTOP-7CGRQ8S:~/LUT_exercises/os_sp/ex1/task1$ ./no_sem & ./no_sem o
[1] 9070
xxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxox[1]+ Done                      ./no_sem
hamidur@DESKTOP-7CGRQ8S:~/LUT_exercises/os_sp/ex1/task1$ ./no_sem & ./no_sem o
[1] 9281
xxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxo[1]+ Done                      ./no_sem
hamidur@DESKTOP-7CGRQ8S:~/LUT_exercises/os_sp/ex1/task1$ ./no_sem & ./no_sem o
[1] 9511
xxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxooooxo[1]+ Done                      ./no_sem
hamidur@DESKTOP-7CGRQ8S:~/LUT_exercises/os_sp/ex1/task1$
```

Figure 1. Terminal showing the mixed output

Task 2

The output here was not freely interleaved like task 1. The characters are in pairs here, for example, xx and oo. So it can be interpreted that one process finished its critical section before the other one entered. However, the overall order of these pairs was still not fixed, and could change from one execution to another.

A named semaphore has been used in this task, and is used as a mutex with an initial value of 1. Each process executed `sem_wait(mutex)` before printing. Thus this prevented the other process from entering the critical section at that same time. That is why the two characters printed by one process stayed together. They produced `xx` or `oo` instead of mixed outputs such as `xo` or `ox`. After printing, the process called `sem_post(mutex)` to release the semaphore. This ensured mutual exclusion, but it did not create a fixed execution order between the two processes. Therefore, the order of `xx` and `oo` groups remained nondeterministic because of process scheduling.

```
hamidur@DESKTOP-7CGRQ8S:~/LUT_exercises/os_sp/ex1/task2$ gcc -o with_sem with_sem.c -lrt
hamidur@DESKTOP-7CGRQ8S:~/LUT_exercises/os_sp/ex1/task2$ ./with_sem & ./with_sem o
[1] 15826
XXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXhamidur@DESKTOP-7CGRQ8S:~/LUT_exercises/os_sp/ex1/task2$ ./with_sem &
./with_sem o
[1]+ Done                               ./with_sem
[1] 16302
XXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXhamidur@DESKTOP-7CGRQ8S:~/LUT_exercises/os_sp/ex1/task2$ ./with_sem &
./with_sem o
[1]+ Done                               ./with_sem
[1] 16574
XXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXX[1]+ Done                               ./with_sem
hamidur@DESKTOP-7CGRQ8S:~/LUT_exercises/os_sp/ex1/task2$ ./with_sem & ./with_sem o
[1] 17059
OXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXX[1]+ Done                               ./with_sem
hamidur@DESKTOP-7CGRQ8S:~/LUT_exercises/os_sp/ex1/task2$
```

Figure 2. Terminal showing compile and run command with resulting output

From Figure 2, it can be interpreted that semaphores can protect critical resources, while preventing interleaving within the critical section. The output became more organized compared to Task 1. This is mainly because each process printed its two characters together. However, the execution order between the two processes still depended on the operating system scheduler.

Task 3

The execution result showed that red always moved first, followed by black. The two processes then continued in a strictly alternating order throughout the execution. On the 10th red move, the program terminated and displayed the message that red won and black lost.

The strict movement order was achieved using two semaphores. The semaphore `hei` was initialized to 1, which allowed the red process to run first. The semaphore `hong` was initialized to 0, so the black process had to wait until red finished its move. After red moved, it posted `hong`, which allowed black to proceed. After black moved, it posted `hei`, which allowed red to continue. This back-and-forth synchronization forced the two processes to move alternately. Since the program was designed so that red wins on its 10th move, black only made 9 moves.

```
hamidur@DESKTOP-7CGRQ8S:~/LUT_exercises/os_sp/ex1/task3$ gcc -o red_chess red_chess.c -pthread -lrt
hamidur@DESKTOP-7CGRQ8S:~/LUT_exercises/os_sp/ex1/task3$ gcc -o black_chess black_chess.c -pthread -lrt
hamidur@DESKTOP-7CGRQ8S:~/LUT_exercises/os_sp/ex1/task3$ ./red_chess & ./black_chess
[1] 20004
Red moves: step 1
Black moves: step 1
Red moves: step 2
Black moves: step 2
Red moves: step 3
Black moves: step 3
Red moves: step 4
Black moves: step 4
Red moves: step 5
Black moves: step 5
Red moves: step 6
Black moves: step 6
Red moves: step 7
Black moves: step 7
Red moves: step 8
Black moves: step 8
Red moves: step 9
Black moves: step 9
Red moves: step 10
hamidur@DESKTOP-7CGRQ8S:~/LUT_exercises/os_sp/ex1/task3$ Red wins. Black loses.
```

Figure 3. Terminal showing compile and run command with resulting output